Automating the Management of Software Systems

WIP - DRAFT

1 Introduction

Software systems are an essential part of modern society and when they fail to function as intended, the implications can be enormous. It is therefore vital that systems are built to minimize the possibility of failure and that, when failures do occur, effective diagnostic processes expedite the systems recovery and prevent the recurrence of failures.

To address this need, this document presents a framework called the Design Learning Platform (DLP) that automates the diagnosis of software failures and fully automates the management of software systems.

2 The Design

A design is a closed semantic world. It contains participants whose state is transformed through the design's unambiguous behaviors. When the environment interacts with the design, it introduces a change to the state of the world. This state transition may enable new behaviors, which the design deterministically resolves in response to the updated state, producing further state changes. When resolving a state, the design may either exhibit behavior directly or deterministically select which behaviors are enabled as a reaction to the state. The design continues reacting to each resulting state transition until no further behavior is enabled and the world reaches a stable semantic state. This chain of resolutions constitutes the structure of the design itself and provides the environment a fully specified path through it.

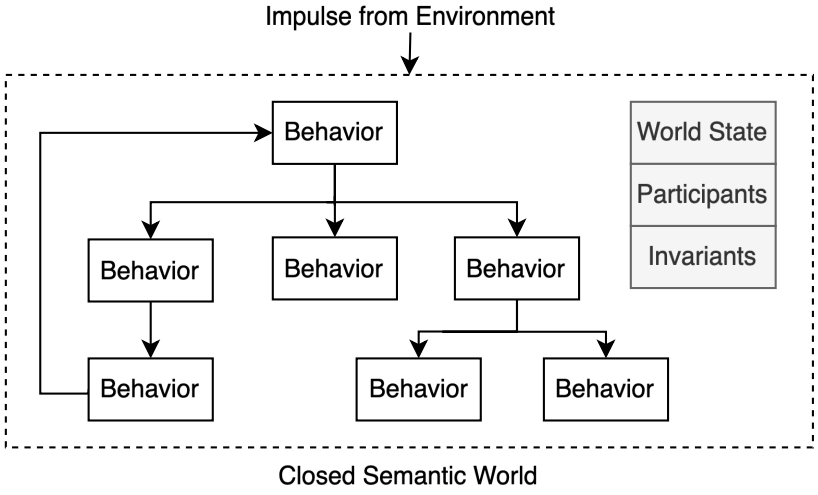

Figure 1: The Design

2.1 Meaning of the World

As part of the design, each behavior has an unambiguous meaning, it unambiguously transforms the participants and its attributes through the transformation. When a behaviors meaning is established as a composition of behaviors, it is called a composite behavior. The composite behaviors meaning is established by the semantics of the behaviors that compose it.

In Figure 2, the meaning of the behavior placeBookOnShelf is composed by the specified behaviors and the established control flow. This is no different than how words work as part of human communication, the word lion is a compression of the meaning of a lion, this includes the appearance, sound etc. A closed semantic world unambiguously establishes the meaning within its world. Through the composition of behaviors, each level of meaning establishes a coherent world.

2.1.1 Shared Meaning

To motivate the need for shared meaning to have successful interactions, consider the following example about a library. A visitor enters the library and finds the book that they want to checkout. They walk up to the librarian with the intention to checkout the book. The librarian greets them with the intention

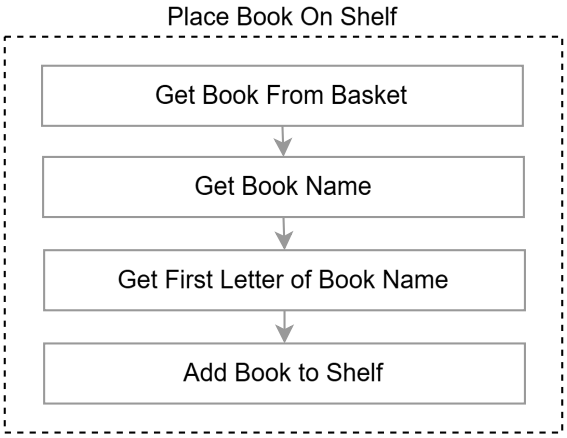

Figure 2: Composite Behaviors

of helping them checkout their book. The user hands them the book but the interaction halts because the librarian was expecting a library card first.

In this example, the interaction failed because of a lack of shared meaning about what it means to checkout a book. Since it is two humans interacting, the librarian can clarify to the user that the correct way to checkout a book is to first provide the library card. The visitor updates their definition of checkout book and the interaction proceeds successfully with shared meaning. This example demonstrates how shared meaning is essential for the success of an interaction.

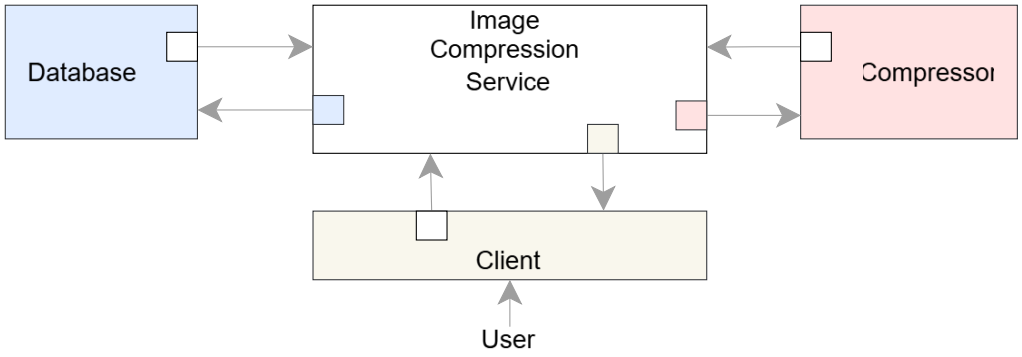

Figure 3: Semantic Compatibility through Shared Meaning

This same principle applies when two designs are interacting, the success of their interaction is dependent on their ability to understand each other. When two designs interact with each other and have shared meaning, the interaction will succeed. Through shared meaning, they form a larger semantic world built through semantically compatible interactions. This process will establish the design of a distributed system assembled through semantically compatible interactions forming a larger closed semantic world with unambiguous meaning.

Figure 3 demonstrates this using a distributed image compression system. For example, when the database is designed, the meaning of the database user is established. When the image compression service interacts with the database, it assumes the role of the database user, thus establishing shared meaning with the database.

The ability to construct larger closed semantic worlds through semantically compatible interactions between designs is meaningful because all the techniques developed in this document naturally extend to the larger world.

2.2 Narrative of the World

Another way of describing a closed semantic world is through the narratives that are established by the design. The narratives are constructed using the behaviors and how it unambiguously modifies the state of the worlds participants. An example of a narrative could be: "The librarian accepted the book from the user and added the book to the basket". This way of speaking about the design captures the meaning of the world in a form that is very intuitive. More importantly, the meaning of the narrative is derived from the constructed world.

2.3 Semantic Invalidity

Semantic invalidity can be generally understood as incorrect or missing meaning. In this sense, the world is in a semantically invalid state when one or more of the following happens:

- The worlds meaning does not satisfy the requirements of the design.
- The intentions of the design cannot be realized due to the state of the world.
- The intentions of the design cannot be realized because reality refuted the designs assumptions.

This means that the correctness of the design can be established by identifying if it eliminates the known semantically invalid conditions. The first step is to ensure that the meaning of the world is correct with respect to its requirements. This is a foundational step because it establishes the correct meaning that the rest of the process builds on.

After ensuring that the requirements are satisfied, when the environment provides an input into this closed semantic world, in order to continue successfully realizing its intentions, the design must prevent semantically invalid states from persisting in its world.

To prevent semantically invalid states from persisting in its reality, semantic invariants act as markers that identify the semantically invalid state and predict the failure of downstream behaviors. This establishes an invariant path from the semantically invalid state to the intention that can't be realized and in order to restore semantic validity, the design must provide a semantically valid path to resolve the state.

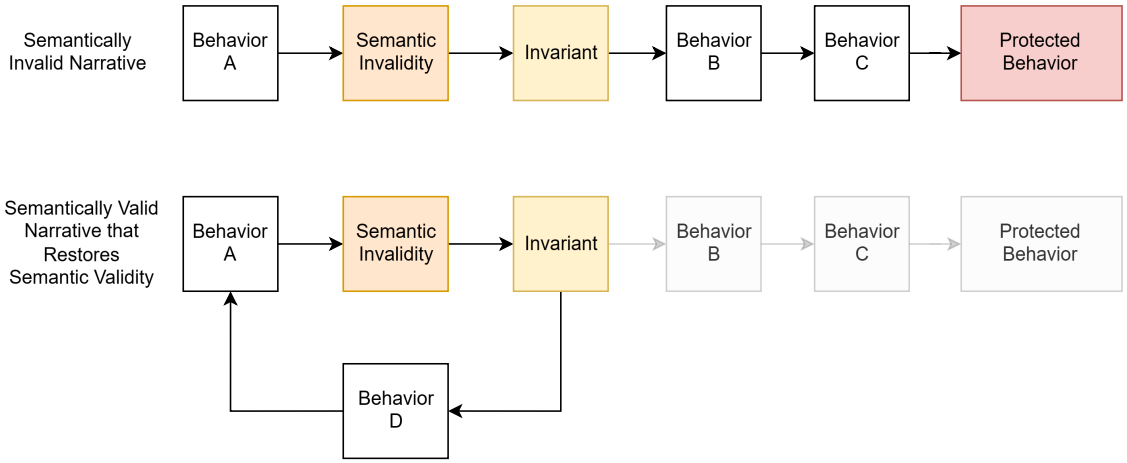

Figure 4: Semantically Invalid Narrative and Semantically Valid Narrative

In order for the closed semantic world to participate in reality, it must be implemented on a substrate. When realized on a substrate, reality can refute the assumptions of the closed semantic word and the design fails because its intentions can't be realized. As a result, the design must learn new semantics and continue to prevent semantically invalid states from persisting in its world to continue realizing its intentions.

In the next section, I will describe how by establishing the design in a structured and unambiguous form and engine can reason about the world. In the process, I will define the world failure and explain how the engine can diagnose failures through semantic reasoning.

3 Semantic Reasoning Engine and Diagnosing Failures

The design is a structure that can be reasoned about through the meaning that it establishes. In this context, reasoning means to ask meaningful questions about it to extract a meaningful answer. As such, if the design is established in a structured and unambiguous form, an engine can automatically reason about the world.

3.1 Meaning of Failures

Reasoning about the world is a familiar process to developers because it has always been a part of software development. This is because software systems have always been the design of a closed semantic world. The process of reasoning about the world is asking meaningful questions about it to get meaningful answers.

When a meaningful question cannot be answered, it exposes missing meaning and in a closed semantic world, this is the meaning of a failure. As such, in this document, the world failure means an inability by the engine to answer a meaningful question about the world.

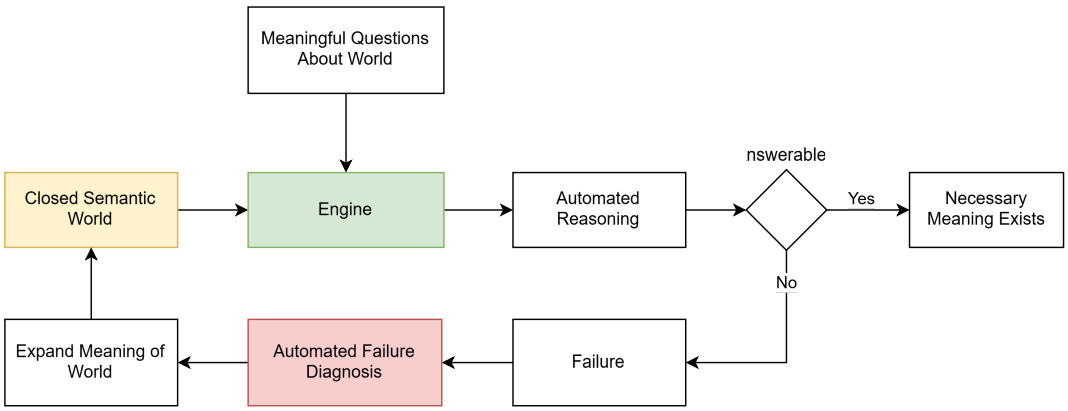

Figure 5: Automating Failure Diagnosis by Reasoning about World to Answer Meaningful Questions

The word failure has historical baggage because it has often been used to reference mechanical failure experienced by the implementation of a design on a substrate (which is one possible result of semantic failure). So I want to make that distinction and clarify that the use of the word failure in this document is not to be confused with mechanical failure. It is worth highlighting this unambiguously because understanding what failures are in a semantic world is the heart of this solution and the way it frames software systems.

In earlier sections, I established the conditions under which semantic invalidity exists. The persistence of the semantic invalidity can be captured through either an inability to answer a meaningful question about the world or an inability to identify the meaningful question entirely. In the following sections, I will describe how semantic invalidity is eliminated through reasoning by expanding the semantics to identify or answer meaningful questions to eliminate failures.

3.2 Design Abstraction Language

To make this possible, the design must be written in a Design Abstraction Language (DAL). The DAL is a declarative language that is used fully establish the closed semantic world. The scope of the DAL includes all the meaning needed to fully define the world (behaviors, participants, attributes, invariants, etc).

3.3 Types of Reasoning

An engine can use the DAL definition to automatically reason about the world. When the engine is unable to answer a meaningful question about the world, the failure is automatically diagnosed because it identifies where the semantics have to expand through root cause analysis.

3.3.1 Reasoning about Requirements

By answering meaningful questions about the requirements, the engine can automatically diagnose failures.

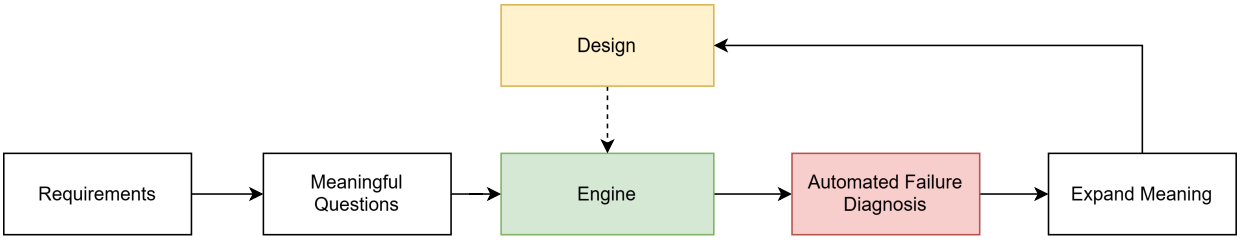

Figure 6: Reasoning to Verify Requirement Satisfaction

3.3.2 Reasoning about Internal Consistency

By reasoning about the world, the engine can eliminate any semantically invalid narratives.

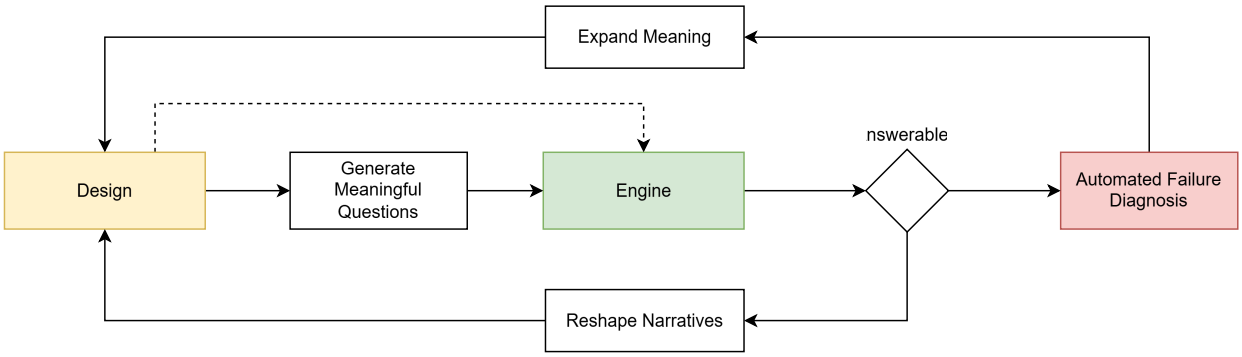

Figure 7: Automated Reasoning to Reshape Narrative

3.3.3 Reasoning about Reality

This means that by reasoning about the execution of the design to identify semantically invalid states, the failure can be automatically diagnosed. This is possible because that the engine can use the meaning of the constructed world to automatically diagnose the failure in the execution and identify where the semantics of the world has to expand through root cause analysis. Through this process, the engine will learn how to answer the meaningful question and eliminate the failure.

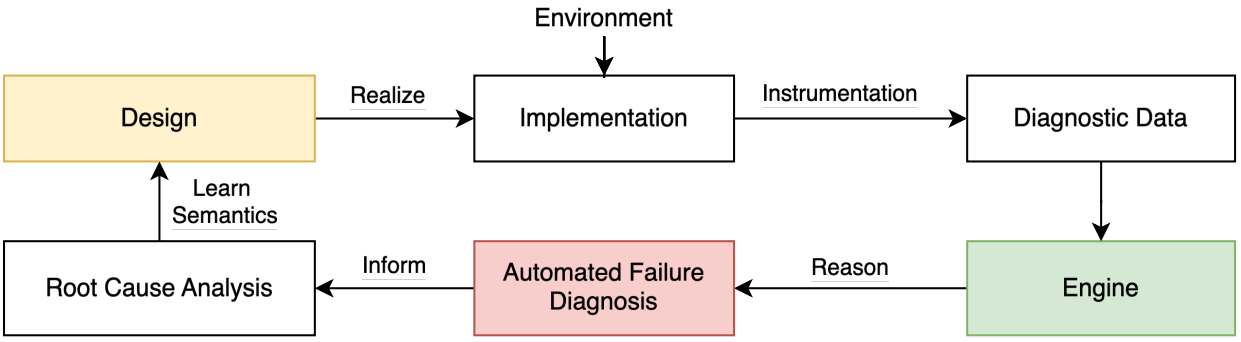

Figure 8: Ideal learning loop by reasoning about execution using design meaning

3.4 Practical Considerations

To do this, the following steps are necessary:

- The implementation and its execution must represent the design semantics faithfully.
- The execution must be losslessly preserved for the reasoning to be deterministic.

If the semantic gap between the implementation and the design is eliminated and the execution is losslessly preserved, it would enable a deterministic learning loop. More importantly, as a result of this, any meaningful question about the world can be automatically answered or an automatic process would collect the necessary diagnostic information needed to learn how to answer the question.

In the following sections, I will explain how a Computable Semantic Model (CSM) can be used to eliminate the semantic gap between the design and the implementation. Then I will explain how domain specific compression can be used to losslessly preserve all the narratives experienced in the world so the engine can deterministically reason about it and finally I will explain how all of this works to fully automate systems management.

4 Computable Semantic Model

In order for the engine to reason about the execution of the design, the semantic gap between the design and the implementation must be eliminated. If this is not done, then the engine cannot deterministically reason about the world because there will be no way to know if the semantics need to expand or if the implementation doesn't realize the meaning of the design.

If the implementation does realize the meaning of the design faithfully, then all the automation that is enabled by the engine reasoning about the design can be leveraged. To make this possible, the design must be established as a Computable Semantic Model (CSM) that is built from Computable Semantic Primitives (CSP) and I will describe how in the following sections.

4.1 Implementing a Computable Semantic Model

A CSM is built by establishing the implementation of the designs meaning enabling the implementation of the design to be directly synthesized into an executable form. The design establishes the first behavior it exhibits and then the design reacts to each impulse from the environment until it reaches a semantically stable state.

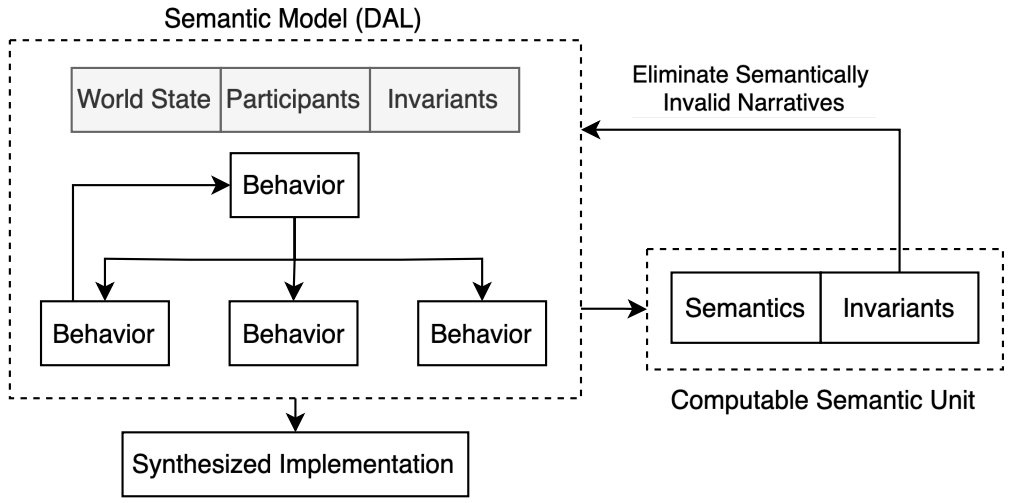

Figure 9: Computable Semantic Model

However, there is still the possibility that the provided implementation doesn't actually realize the meaning of the design, in which case, the engine cannot reason about the world in a reliable way. Moreover, the engine can only reason about the world based on the meaning that was specified. So unless the design makes all the relevant semantics available, the engine will be unable to determine the worlds correctness. So the engine will know that there is a failure and it won't be able to explain why until it learns the missing semantics.

In the next section, I will describe how designs built from Computable Semantic Primitives addresses both of these problems.

4.2 Computable Semantic Primitives

Computable Semantic Primitives (CSP's) are primitive computable semantic units that can be used to construct the meaning of the semantic world. Since the world is constructed from meaningful atoms that have unambiguous invariants, it can be reasoned about completely to eliminate all semantically invalid narratives. Since the primitive units can be synthesized into an implementation by definition, this transforms the DAL into a declarative language that establishes the meaning of a closed semantic world and synthesizes the meaning of that world as an implementation, effectively making the design itself executable.

As previous sections mentioned, the engines ability to reason about the execution of the world is dependent on there being no semantic gap between the design and the implementation. Using CSP's solves this problem elegantly in multiple ways:

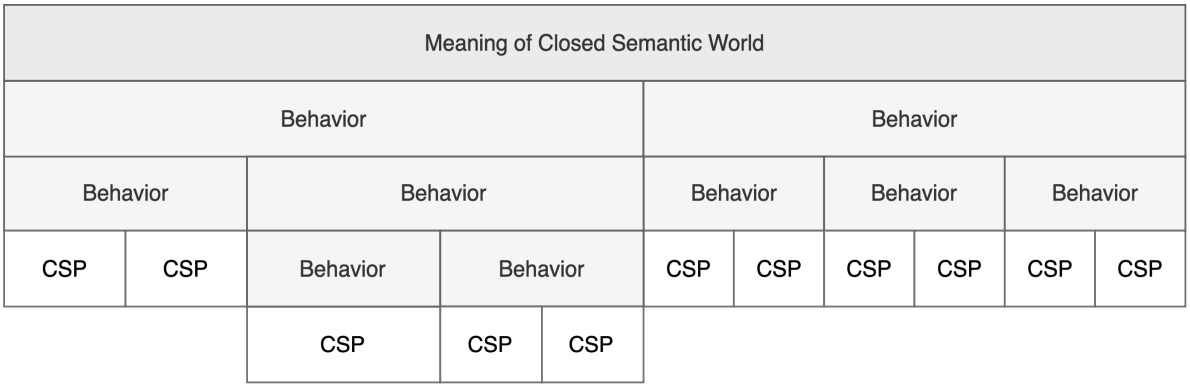

Figure 10: Computable Semantic Model constructed from Computable Semantic Primitives

1. It allows the engine to fully reason about the constructed world to eliminate any semantically invalid narratives.
2. It allows the engine to reason about the execution to automatically diagnose any failures and motivate the necessary root cause analysis to learn new semantics.

Now that the constructed model and the execution have the same meaning, the engine can automatically identify where the semantics need to expand because reality will refute the designs assumptions through failures.

However, this automation will still not be possible unless the engine actually has a complete account of the execution to diagnose the failure. In the next section, I will talk about the information that needs to be preserved to make this possible.

4.3 Automated Instrumentation

Since the design is an unambiguous structure, it isn't necessary to preserve all the execution information to be able to replay it. The design itself can identify the information that can't be deterministically computed by the design. This means that the synthesized implementation can be automatically synthesized to only instrument the non-deterministic information.

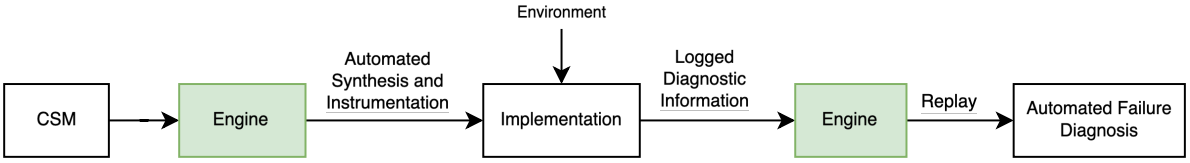

Figure 11: Automated Instrumentation and Synthesis

As Figure 11 illustrates, the engine can use the unambiguous specification of the CSM to identify the necessary instrumentation. The information produced by the execution is then fed back into the engine where the failure is automatically diagnosed. In this way, the ways in which reality refutes the assumptions made by the design can be unambiguously identified and the learning process can be triggered.

However, to truly make this an automated process, all the diagnostic data needs to be preserved at scale. Any missing data would introduce speculation, so there is a need for a solution address the practical problem while minimizing cost. In the next section, I will describe how this can be addressed by using the unambiguous domain structure of the data to apply domain specific compression.

5 Domain Specific Compression

In order to preserve all the diagnostic data necessary for the engine to deterministically reason about the world, the unambiguous domain structure of the CSM can be leveraged to apply Domain Specific Compression (DSC). This technique leverages the known domain structure to optimally compress it.

5.1 Compressed Log Processor

Practically, this can be achieved using a tool named Compressed Log Processor (CLP). It is an open source tool that can apply DSC and search the compressed data without decompression. More importantly, it has proven that DSC is practical at a petabyte scale. Currently, CLP compresses structured and unstructured logs by exploiting their known domain structure. However, in its current form, the domain structure of the variables in those logs is not know, so CLP is not maximizing compression it can achieve.

Figure 12: CLP Logo

Using the CSM, CLP can be transformed into a DSC engine. CLP's gains come from a lack of ambiguity and the CSM works to eliminate ambiguity entirely in its domain, thus CLP's compression will be maximized. More importantly, since the data exists as part of a structure with unambiguous meaning, the compression will improve further by exploiting the repetitive and predictable nature of programs.

When the engine replays the execution using the collected diagnostic data to automatically diagnose the failures, the narratives that are played out in the semantic world can be preserved by CLP at minimal cost. This means that the engine will never have to re-compute the same transformation twice. Instead, it can leverage CLP's search without decompression feature to identify if the narrative's transformation has already been computed.

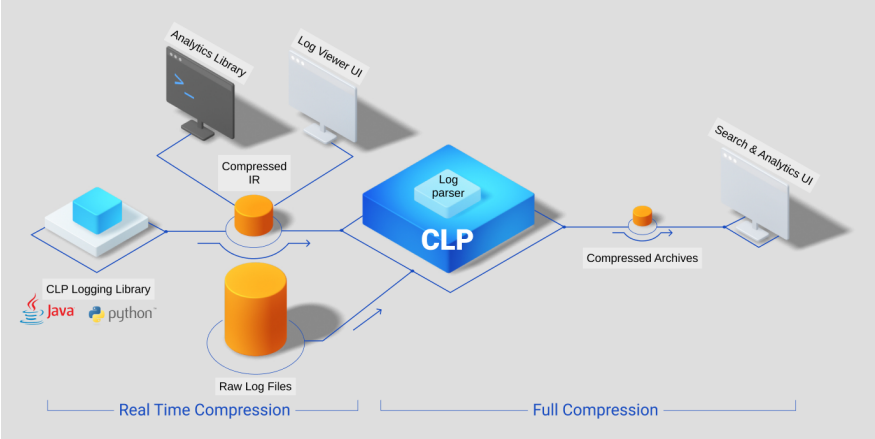

Figure 13: Overview of CLP's current solution.

This creates a very powerful ability where CLP contains the entire experiential history of the closed semantic world. When combined with the CSM, this is what allows the engine to reason about the world deterministically. CLP's search performance will also become more optimized since the both the structure and the meaning of the data are entirely unambiguous. The queries themselves are inherently semantic because the data exists as part of a closed semantic world and the queries can be meaningfully structured.

Ultimately, this will result in a fully optimized platform because the meaning of any data in the platform is inherently known. Moreover, when CLP is combined with the CSM and the engine, this results in a design learning platform that fully automates software systems management.

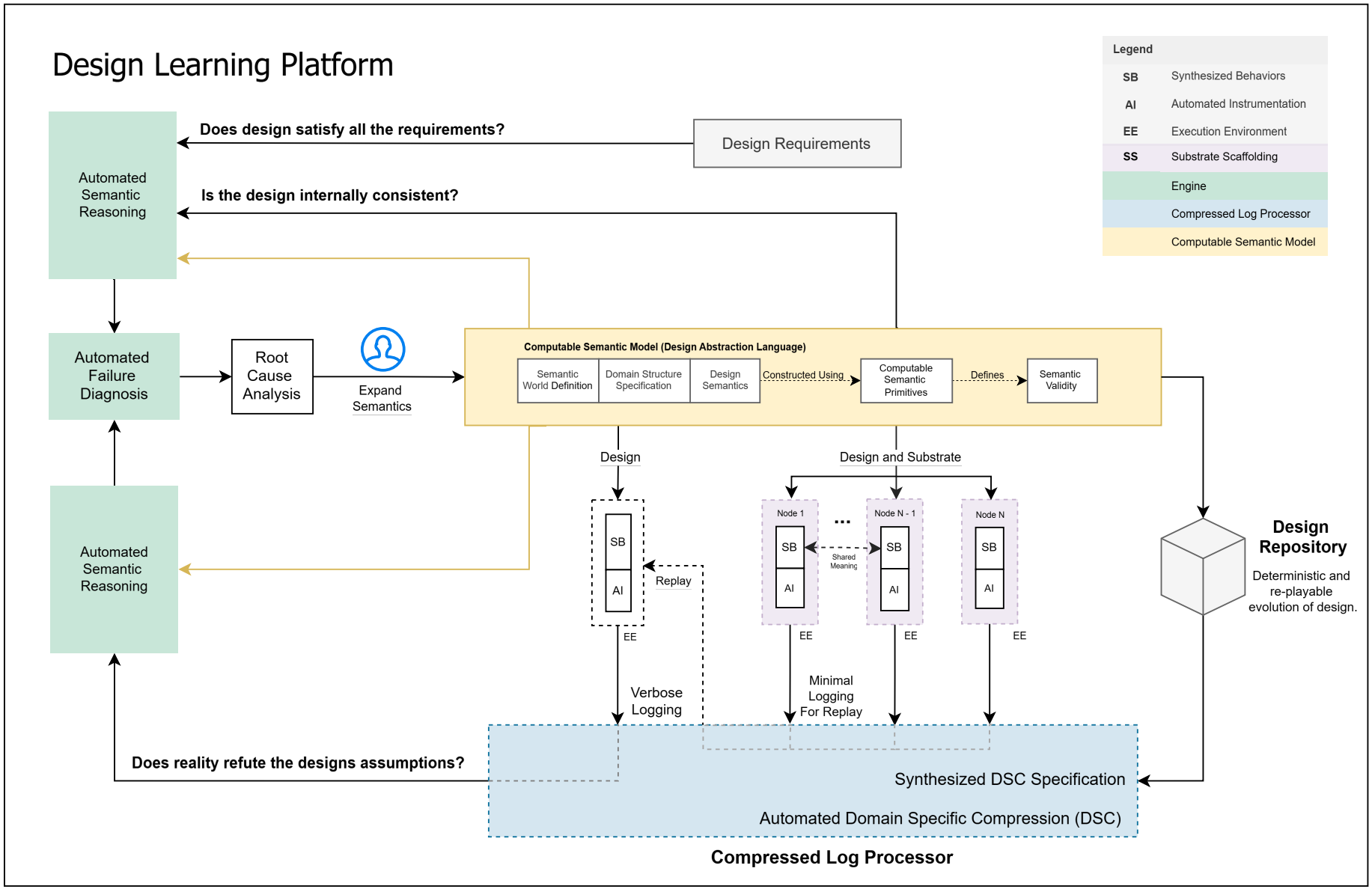

Figure 14: The Design Learning Platform

6 Design Learning Platform

In this section, I will present the Design Learning Platform (DLP) and describe how it fully automates the management of software systems. Many of the ideas presented in this section have been established in earlier sections but this section brings them all together under one tool to fully automate systems management.

6.1 Automated Systems Management

Systems management can be described as the process of reasoning about a system or its state to take the appropriate action. In developing the computable semantic model, first, the structure that can be automatically reasoned about using an engine was established. Next, the semantic gap between the implementation and the Computable Semantic Model (CSM) was eliminated using Computable Semantic Primitives (CSP's), allowing the same engine to reason about the execution of the CSM. This enabled the engine to reason about the world, synthesize/instrument the implementation and to reason about the execution of the world. Finally, this process was enabled at scale by using Compressed Log Processor (CLP) to losslessly retain the narratives experienced by the world and to efficiently diagnose the failures from the observed narratives.

By establishing these capabilities, any meaningful question about the world can be answered, given that the necessary meaning has been established. If it cannot be answered, the CSM provides the mechanism to eliminate the ambiguity to be able to answer the question automatically. All of the automated management steps I will describe below are a result of this lack of ambiguity and can ultimately be classified as a form of semantic reasoning.

The Design Learning Platform is a solution that leverages these capabilities to fully automate systems management. I will first describe a how the reasoning is applied to automated systems management in a few specific ways and then I will describe how semantic reasoning can be applied in a general way by posing the right question and establishing the necessary meaning to answer that question.

6.1.1 Deterministic Learning through Automated Failure Diagnosis

At the core of this tool is the CSM which specifies a closed semantic world built from CSP's and is written in the Design Abstraction Language. The engine then reasons about the closed semantic world and ensures that every semantically invalid narrative is eliminated by establishing a semantically valid narrative that prevents the semantic invalidity. Then the engine tests that every semantically invalid narrative is eliminated by walking the invalid narrative path and verifying that the design selects a semantically valid narrative.

Then the CSM is synthesized and instrumented to collect the necessary diagnostic data to enable replay the execution. This results in the engine automatically diagnosing observed failures because the existence of a semantically invalid narrative means that the design must learn new semantics to predict and resolve the failure.

Then the same testing process repeats and through testing every semantically invalid narrative is eliminated and the design learns new semantics when a failure is observed. In this process, failure diagnosis, testing and ultimately debugging become automated.

All of this automation is simply a result of automatically reasoning about the world. If the necessary meaning was not established in the world, the engine would not be able to automatically reason about it. The failure to learning loop is an application of this principle, the engine can't explain why the design failed, so new semantics are learnt.

In the next section, I will speak more generally about how automated reasoning can be used to manage the system.

6.1.2 Automated Reasoning

Ultimately, the processes described in the previous section are a form of automated reasoning. Invariant placement, testing, failure diagnosis are all questions that can be asked about the world that the engine can answer. In fact, this way of framing things makes this process a lot more intuitive. What other meaningful questions can be asked about the world and in what way can ambiguity be eliminated to make it possible to answer this question?

Consider the question which reasons about whether the design satisfies all the requirements of the design. This question is a good one because it expands what is included in systems management. This question can be automatically answered if the requirements and the design share meaning. Interestingly, the requirements will populate a level of meaning in the closed semantic world by construction and this will establish the shared meaning with the design. So for example, one of the requirements says that if a book is submitted to the library with name X, then it must be accessible using the same name X.

In this example, the question can be answered by the engine reasoning about the meaning of submit and access in the closed semantic world. If the design provides a mechanism to submit book named X and then access book named X, then the requirements are satisfied. This doesn't tell you if the book was actually placed on the shelf and and accessed from the same shelf. There will be another set of requirements that establish this capability and that can be verified independently. In this sense, the under specification of the requirements can actually result in the meaning of world being incompletely defined. However, this is another example of how the engine can reason about a world and answer meaningful questions about it to automatically answer it. In this case, if the engine can't answer if the requirements are met by the design, then the design needs to evolve.

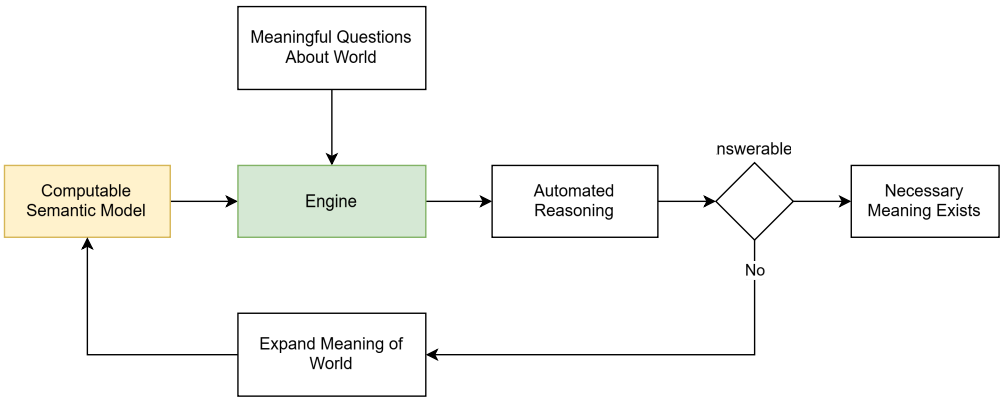

Figure 15: Establish necessary meaning by answering meaningful questions through semantic reasoning.

In this way, the questions that can be asked about the world ensure that the necessary meaning exists to be able to answer it automatically. This is a powerful idea because the meaningful questions establish the kinds of reasoning that is necessary and this in turn establishes the necessary meaning in the world. Automated management becomes establishing all the questions that you need to be able to answer about the world by reasoning about it using the engine and then establishing the meaning needed to make it happen. Eventually, there will be a set of questions which act as proof that the world has established the necessary meaning.

Until now, I was talking about how the correctness of the world itself can be identified through automated reasoning. However, there are other meaningful applications of this kind of reasoning to improve the efficiency of the world. For example, the engine can automatically identify if narrative branches are redundant. This is possible because it can identify if independent branches or sequence of behaviors have no meaningful effect on the world and remove the redundancy to make the world more optimized. It establishes if every narrative in the world is necessary or if it can be simplified. These are questions that the engine can ask about the world on its own to reshape the world, similar to how it eliminates the semantically invalid narratives.

I hesitate to conclude this section because there are so many more examples of how this can be applied. I particularly enjoyed the questions posed by the requirements because it exposes how semantic reasoning can be used to establish the necessary meaning in the world to make sure the design realizes its meaning. In the same way, there are many questions that can be posed, the system will be managed by asking the necessary questions that the world must be able to automatically answer and establishing the meaning necessary to answer the question.

6.1.3 Automated Orchestration

Automated orchestration falls directly from the ability to build computable semantic models which have shared meaning. When a distributed system is defined, it also defines an orchestrator that has shared

meaning with the design. It is through this shared meaning that orchestration is made automated and surgical because the state of the system being managed is unambiguous by design.

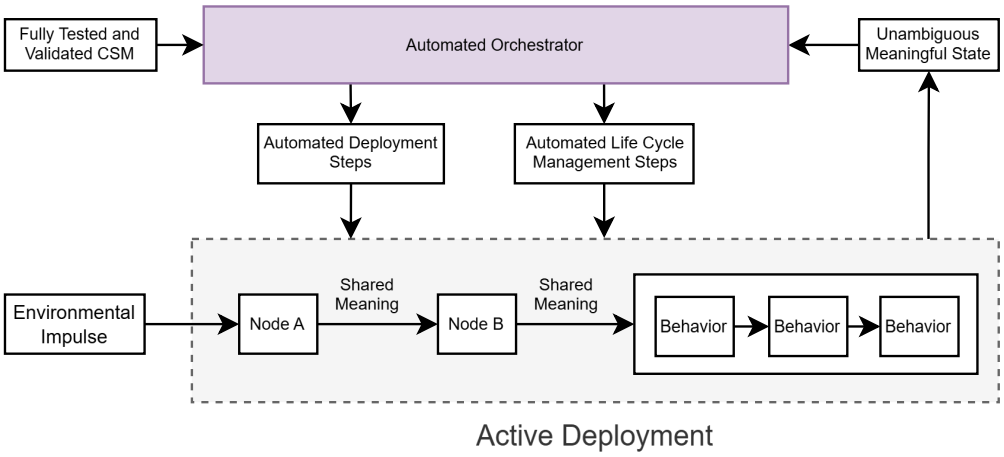

Figure 16: Automated Orchestration of Distributed System

Figure 16 shows shows a block diagram how of orchestrator can react to an unambiguous meaningful state to take the precise response because it knows the meaning of the CSM that it is managing by design.

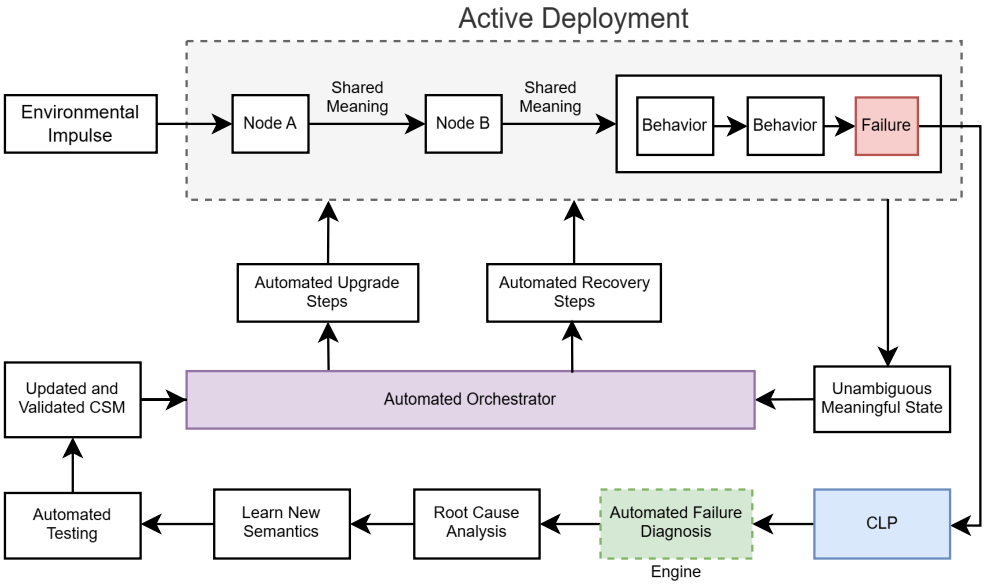

Figure 17: Automated Orchestration of Recovery from Mechanical Failures

Figure 17 shows a block diagram of how an orchestrator can react to a failed state and take the necessary steps to recover the system. This is once again made possible because the failed state has unambiguous meaning to the orchestrator by design. At the same time, the learning loop described in a previous section works to ensure that the failure is automatically diagnosed and a deterministic learning process prevents the recurrence of the same failure.

Ultimately, this isn't anything special, it is just an extension of the closed semantic world to include the meaning which establishes the conditions needed for the design to realize its intentions. Since each part of this system will be connected together using semantically compatible interactions, the same principles that establish the correctness of a design apply here. This also means that the testing of the orchestrators response to meaningful states in the design can be automatically tested.

6.2 Design Repository

Traditionally a code repository contains the evolution of the implementation of the program and the meaning of the evolution is limited to the commit messages or the pull requests that describe the commit.

However, with this solution, since every environment that motivated new semantics is preserved, it results in a design repository through which the evolution of the software system can be deterministically replayed. The repository tracks not just the changes to the implementation but also how the system was designed, executed and refined through learnt semantics over time.

This is a natural conclusion of eliminating any ambiguity in the construction and of a software system and in the execution of that constructed system, it eliminates ambiguity in its evolution.

7 Additional Thoughts

In this section, I will discuss some additional thoughts that I couldn't naturally add in the rest of the document but that I think are relevant to this framework.

8 Securing Interactions

As a result of this framework, interactions between and within designs now has unambiguous meaning. There are meaningful implications for securing systems because the intention of the interaction is unambiguous. While before, the mechanical sequence had to be correct, now the design can ensure that it is the right person that is communicating with it. If the design is looked at as the brain of the software system, then this ensures that the correct brain is speaking, making it more difficult to compromise a system.

There are many approaches to implement this but one possibility is that DAL can be extended to generate a dynamic checksum along the narrative path to identify the unique brain that generated the narrative. Unlike traditional techniques which have to speculate about whether the intentions of a particular sequence are suspicious through practices like anomaly detection, this solution would automatically recognize suspicious intentions from construction. The security is is built into the narratives that are permitted in the closed semantic world.

There are also automated ways to test and verify how secure a system is by following the same principle of eliminating ambiguity using the CSM to answer meaningful questions about the system. A future where software systems understand each others intentions and meaning will be a more secure future.

9 Automated Documentation

Traditionally, documentation was the developers best attempt to capture the design of the system. This was then used by other developers to establish the necessary semantics to interact with the design correctly. It is clear that this relies on the skill of the developer heavily and can introduce bugs through human error.

Through the processes established in this document, the documentation has become automated because the design is able to represent itself. More importantly, the designs themselves can verify that there is semantic compatibility between each other by ensuring that the necessary meaning has been established on both sides. In this sense, the documentation is the design and since it is representing itself, the documentation itself is used to verify the correctness of the semantics.

By enabling the CSM to represent itself unambiguously as the documentation, software systems are made more reliable because it minimizes the possibility of errors.

10 Research

In this section, I will briefly discuss this novelty of this solution from a research perspective. It is difficult to call the CSM a breakthrough because it is simply formalizing the existing software engineering practices into a computable model so that it can be automatically reasoned about by an engine. However, it is novel in the sense that this capability did not exist before and it is a necessary capability to automate systems management.

Domain specific compression is also a necessary capability to make the automation possible because it overcomes a hurdle that prevented automated systems management. Until CLP existed and proved that domain specific compression was possible at a petabyte scale, there was no solution that could address this need or even establish that the problem was solvable. In this sense, the novelty of CLP is that it proved that the preservation of the information necessary to automate systems management is possible at scale. So from a research perspective, the maturity of domain specific compression was a real breakthrough that made automated systems management possible.

However, neither the CSM, the engine or CLP solve this problem on their own. They all exist as part of a coherent solution that works to automate systems management. By transforming the development of software systems into a process of meaningful world building, this lays a meaningful foundation that opens the door for many new areas of research.

11 Conclusion

The techniques presented in this document take a fundamentally different approach to software systems management than existing approaches. It provides the means to complete the construction of a software system by establishing its design in a Design Abstraction Language (DAL) as a Computable Semantic Model (CSM) built from Computable Semantic Primitives (CSP) that unambiguously establishes its meaning and eliminates any semantic gap between the design and its implementation

This results in an engine that can automatically reason about the closed semantic world to answer any meaningful question given that the necessary meaning has been established. It also enables the engine to automatically reason about the execution of the software system to automatically diagnose failures and extend the semantics of the world. In order to fully automate systems management, a proven tool named CLP is leveraged to preserve meaningful history of the design using domain specific compression.

Rather than asking what information is needed to diagnose a particular failure, using CLP and the CSM, this preserves all the information needed by the engine to reason about the software system completely. This makes this solution particularly powerful because the CSM provides the means to define the meaning necessary to answer any meaningful question about the world. The applications mentioned in this document are just a form of reasoning to automate the management of software systems, in my opinion, the potential applications are only limited by the questions that can be asked about a closed semantic world.

Ultimately, this solution does not invent anything new. Software systems have always operated at this level of meaning and there is nothing that this solution presents that wasn't already common knowledge. What makes it novel is that it takes steps to unambiguously understand the implementation of the software system through the meaning established by its design and enables an engine to automatically reason about it.

This solution ultimately eliminates ambiguity in the development and maintenance of software systems. As a result, the software systems that run modern society are less likely to fail and if they do fail, an automated process ensures their recovery and automated failure diagnosis supports deterministic learning to prevent future failures. This framework transforms software development into a meaningful process of world building and lays the foundation for new and exciting research. The future of software development will be less about writing bug free code and more about building intelligent and secure systems that understand each other.

12 Extended Applications

In this section, I present some applications that can immediately benefit from this framework outside of software systems and also some future thoughts for interesting applications that I would want to explore within the world of software systems.

12.1 Universal Design Language and Digital Twins

Designs are the universal structures used across many disciplines. The ability to unambiguously establish a closed semantic world and reason about it has applications well beyond software systems. If implemented with the right strategy, this framework has the ability to become a universal design language used in many disciplines.

Since the information contained in the DAL has unambiguous meaning, it can be used to establish a powerful visualization framework. This would allow engineers to visualize the closed semantic world in a meaningful way and gain insights into their design by reasoning about its meaning. More importantly, since this framework extends the design into a computable semantic model (CSM), it extends the CSM to become a digital twin of what it is modelling.

In software systems, the design establishes a closed semantic world that mirrors what participates in reality because the design is what defines it (this statement is even more valid with CSM's constructed using CSP's). This is a unique property of software systems since they operate entirely on a stack that is symbolically defined by humans. However, in other disciplines, reality is modelled by engineers to be able to represent it accurately and predict it. In these applications, the CSM becomes the framework through which that modelling happens.

In the digital twin models, information collected by sensors is used to inform the model and refine it so that it is a more accurate representation of reality and to improve its ability to make more accurate predictions. In these cases, the domain structure of the collected data is unambiguously established by the CSM, as such, the data can be domain compressed using CLP's engine. This extends the use of CSM and CLP beyond software systems.

The applications of CSM are only limited by what can be modelled, anything that can be modelled as a closed semantic world can be reasoned about using the engine. A personal favorite of mine is to model concepts using the CSM so that the model can be used as a teaching tool. This applications exploits the fact that when humans teach, they are ultimately building a closed semantic world to convey the concept to another human mind. Using the CSM, the semantic world necessary to represent the idea can be transformed into a computable model in which every step in the process is informed and validated by the model. There are many interesting ways in which this can be used to turn teaching and learning into an unambiguous process that converges on complete understanding.

12.2 Future Exploration: Automated Development

The thoughts in this section are more future oriented but something that I would enjoy exploring. I believe that the CSM built from CSP's opens the door to interesting approaches for automating the development of software systems because the engine has the ability to reason about the world to identify its internal consistency and correctness. I think there are interesting techniques where a semantic toolkit is used to construct closed semantic worlds and expand the meaning of world.

Since failure diagnosis is automated and the design can deterministically learn new semantics through root cause analysis, I think there are applications where the design operates entirely in a virtual environment and integrates its meaning into the existing closed semantic world. In a solution like this the simulated environment itself has unambiguous meaning and when the design fails, it learns how it must modify its behavior to achieve its intentions in its environment.

It would be able to perform root cause analysis on the failure because the entire world is modelled and the meaning is unambiguous. While there are certainly many intermediate steps on the way to reaching this, I feel that when software systems are modelled in this way to establish its meaning as a computable model and they start to work together in a simulated world, solutions like this will become more realistic where designs try and fail in simulations to learn.